



MANUAL TÉCNICO

Ingrediente 791

Plataforma web de recetas de cocina casera

Plataforma web para la publicación y gestión de recetas de cocina, con sistema de usuarios, interacción social en tiempo real y panel administrativo.

Versión del documento	1.0
Versión del sistema	0.1.0
Stack tecnológico	Next.js 14 + Supabase + TypeScript
Tipo de documento	Manual técnico de sistema
Fecha de emisión	Mayo 2026

Contenido

1. Introducción	7
1.1 Propósito del documento	7
1.2 Alcance	7
1.3 Audiencia objetivo	7
1.4 Convenciones utilizadas	8
1.5 Características principales	8
2. Descripción general del sistema.....	9
2.1 Visión del producto	9
2.2 Funcionalidades principales.....	9
2.2.1 Módulo de autenticación	9
2.2.2 Módulo de recetas.....	9
2.2.3 Módulo social	10
2.2.4 Módulo de blog	10
2.2.5 Módulo administrativo	10
3. Arquitectura de la aplicación	11
3.1 Patrón arquitectónico.....	11
3.2 Capas del sistema	11
3.3 Diagrama de arquitectura	12
3.4 Regla de clientes Supabase	13
4. Stack tecnológico.....	14
4.1 Frontend	14
4.2 Backend como servicio (BaaS).....	15
4.3 Servicios externos	15
4.4 Requisitos de entorno.....	15
5. Estructura del proyecto	16
5.1 Organización de carpetas	16
5.2 Archivos clave	17
5.2.1 middleware.ts.....	17
5.2.2 lib/supabase-server.ts	17
5.2.3 lib/supabase-browser.ts	17

5.2.4 types/index.ts	17
6. Modelo de base de datos	18
6.1 Visión general.....	18
6.2 Diagrama entidad-relación.....	19
6.3 Descripción detallada de las tablas.....	20
6.3.1 profiles	20
6.3.2 recipes	21
6.3.3 likes.....	22
6.3.4 comments	23
6.3.5 favorites	24
6.3.6 posts	25
6.3.7 post_likes y post_comments	25
6.3.8 contact_messages	26
6.3.9 site_settings	26
6.4 Índices	27
6.5 Triggers	27
7. Políticas de seguridad (RLS).....	28
7.1 Introducción a Row Level Security.....	28
7.2 Matriz de políticas.....	28
7.3 Ejemplos de políticas.....	29
7.3.1 Política de lectura pública	29
7.3.2 Política de propiedad	29
7.3.3 Política restrictiva (favoritos)	30
7.4 Validación adicional en la aplicación.....	30
8. Módulos funcionales	31
8.1 Módulo de autenticación	31
8.1.1 Registro	31
8.1.2 Login	31
8.1.3 Recuperación de contraseña	31
8.2 Módulo de recetas	31
8.2.1 Listado y búsqueda	31
8.2.2 Detalle de receta	32
8.2.3 CRUD administrativo.....	32
8.3 Módulo social	32
8.3.1 Sistema de likes	32
8.3.2 Sistema de favoritos.....	32

8.3.3 Comentarios en tiempo real	32
8.4 Módulo de blog.....	33
8.5 Módulo administrativo.....	33
9. Gestión de archivos (Storage).....	34
9.1 Buckets definidos	34
9.2 Políticas de acceso a Storage	34
9.3 Convenciones de nombres	34
9.4 Formato obligatorio: WebP	35
9.5 Límites de tamaño	35
10. Autenticación y middleware.....	36
10.1 Flujo de sesión	36
10.2 Código del middleware	37
10.3 Protección CAPTCHA.....	38
11. Sistema de autenticación	39
11.1 Flujo general	39
Pasos del registro con correo	39
11.2 Cliente de servidor: supabase-server.ts	40
11.3 Cliente de navegador: supabase-browser.ts	41
11.4 Middleware: refresco automático de sesión.....	42
11.5 Callback de email: app/auth/callback/route.ts	43
11.6 Verificación de admin.....	44
12. Seguridad	45
12.1 Capas de defensa.....	45
12.2 Row Level Security (RLS)	45
Ejemplos de políticas críticas	46
12.3 Cookies seguras.....	46
12.4 Variables de entorno.....	46
12.5 Captcha en registro.....	47
13. Sistema de búsqueda	48
13.1 Arquitectura del buscador.....	48
13.2 API de búsqueda: /api/search/route.ts	48
Lógica de búsqueda.....	48
13.3 Componente LiveSearchBar.....	50
Características técnicas clave.....	50
Hook principal: fetchSuggestions	51
Handler de cambio de input con debounce	52

Navegación por teclado	53
13.4 Caché de búsqueda: SearchDropdown.tsx.....	54
13.5 Buscador de la página /recetas (Server Component).....	55
14. Componentes principales	56
14.1 RecipeCard	56
Props	56
Características	56
14.2 LikeButton — Optimistic Updates	57
14.3 FavoriteButton	59
14.4 CommentSection — Comentarios en tiempo real	59
Suscripción Realtime	59
Envío de comentario o edición	60
14.5 Navbar.....	61
Suscripción al estado de autenticación	61
14.6 AdminPanel	62
Estructura del estado	62
15. Funciones utilitarias clave	63
15.1 slugify — Generar slug de URL	63
15.3 uploadFile — Subir archivo a Storage	65
15.4 convertToWebP — Conversión de imagen en cliente	66
15.5 Extracción automática de miniaturas de YouTube	67
16. Panel administrativo.....	68
16.1 Acceso	68
16.2 Estructura del panel	68
16.3 Flujo de creación de receta	69
16.4 Validación de imágenes WebP.....	69
16.5 Carga de datos en servidor.....	70
17. Rutas y navegación.....	71
17.1 Mapa de rutas.....	71
17.2 Grupo de rutas (main).....	71
17.3 Rutas dinámicas	72
17.4 Server Components vs Client Components.....	72
18. Guía de instalación y despliegue.....	73
18.1 Preparación del entorno local	73
18.1.1 Clonar el repositorio	73
18.1.2 Instalar dependencias	73

18.2 Configuración de Supabase.....	73
18.2.1 Crear el proyecto.....	73
18.2.2 Obtener las credenciales.....	73
18.2.3 Archivo .env.local	73
18.2.4 Ejecutar el esquema SQL	74
18.2.5 Configurar proveedores de autenticación	74
18.3 Primer administrador	75
18.4 Ejecución en desarrollo	75
18.5 Despliegue en producción	75
18.5.1 Despliegue en Vercel	75
18.5.2 Compilación manual.....	75
19. Estilos y diseño	77
19.1 Paleta de colores: Warm Linen.....	77
19.2 Tipografías	77
19.3 Clases utilitarias del proyecto	78
20. Mantenimiento y solución de problemas	79
20.1 Problemas frecuentes.....	79
20.1.1 La sesión no persiste o se cierra sola	79
20.1.2 Error 'relation does not exist'	79
20.1.3 Las imágenes no cargan	79
20.1.4 Error 401 / No autenticado	79
20.1.5 Fallo al publicar recetas	79
20.2 Tareas de mantenimiento periódicas	80
20.3 Logs y depuración	80
21 Comandos de diagnóstico Supabase.....	81
22. Glosario técnico	82

1. Introducción

1.1 Propósito del documento

El presente manual técnico describe de manera detallada la arquitectura, componentes, estructura de base de datos, políticas de seguridad y procedimientos de despliegue del sistema Ingrediente 791, una plataforma web desarrollada para la publicación de recetas gastronómicas y la interacción social entre usuarios.

Este documento está dirigido al personal técnico responsable del mantenimiento, evolución y despliegue de la aplicación: desarrolladores, administradores de base de datos, ingenieros DevOps y líderes de proyecto.

1.2 Alcance

El manual abarca la totalidad del sistema Ingrediente 791, incluyendo:

- **Frontend:** aplicación Next.js 14 con App Router, Server Components y Client Components.
- **Backend como servicio:** integración completa con Supabase (PostgreSQL, Auth, Storage, Realtime).
- **Base de datos:** esquema relacional con doce tablas principales, triggers, índices y políticas RLS.
- **Almacenamiento de archivos:** buckets de Supabase Storage para imágenes y videos.
- **Seguridad:** Row Level Security en todas las tablas, validación de CAPTCHA mediante Cloudflare Turnstile, y manejo de sesiones por cookies HTTP-only.

1.3 Audiencia objetivo

Este documento presupone conocimientos intermedios en desarrollo web moderno, bases de datos relacionales (PostgreSQL) y despliegue de aplicaciones en plataformas cloud. Se recomienda familiaridad previa con React, TypeScript y los servicios de Supabase.

1.4 Convenciones utilizadas

A lo largo de este manual se emplean las siguientes convenciones tipográficas:

- **Código fuente:** se presenta en fuente monoespaciada sobre fondo gris claro.
- **Nombres de archivos y rutas:** se muestran en fuente monoespaciada.
- **Comandos de terminal:** precedidos del símbolo \$ cuando corresponde.
- **Notas importantes:** destacadas en bloques con fondo coloreado.

1.5 Características principales

- Autenticación completa con correo electrónico y Google OAuth, incluyendo verificación de captcha en el registro.
- Sistema de recetas con CRUD completo desde el panel administrativo, soporte para imagen principal, miniatura optimizada y video embebido de YouTube.
- Buscador en vivo con sugerencias instantáneas, normalización de acentos y búsqueda en título e ingredientes.
- Sistema de likes y favoritos con actualizaciones optimistas (optimistic updates) para una experiencia fluida.
- Comentarios anidados con actualizaciones en tiempo real mediante Supabase Realtime.
- Blog secundario (Posts) con imágenes en carrusel, video de YouTube, likes y comentarios.
- Panel administrativo con vista general (overview), gestión de recetas, mensajes de contacto y publicaciones del blog.
- Diseño responsive mobile-first con paleta de colores cálida personalizada (Warm Linen).
- Seguridad a nivel de fila (Row Level Security) en todas las tablas mediante políticas RLS de PostgreSQL.

2. Descripción general del sistema

2.1 Visión del producto

Ingrediente 791 es una aplicación web moderna pensada como plataforma oficial del canal gastronómico Ingrediente 791. Su objetivo es ofrecer un espacio digital donde el chef pueda publicar recetas acompañadas de fotografías y videos, y donde la comunidad de usuarios pueda interactuar mediante likes, comentarios, favoritos personales y mensajes de contacto.

El sistema adopta un modelo de contenidos administrados: solo los usuarios con rol de administrador pueden publicar recetas, mientras que cualquier usuario registrado puede interactuar con ellas. Los visitantes no autenticados pueden consultar todo el contenido público pero no pueden comentar, dar like ni guardar favoritos.

2.2 Funcionalidades principales

2.2.1 Módulo de autenticación

- Registro de usuarios mediante correo electrónico y contraseña.
- Autenticación federada a través de Google OAuth.
- Recuperación de contraseña por enlace enviado al correo.
- Protección mediante Cloudflare Turnstile (CAPTCHA invisible) en el formulario de registro.
- Sesión persistente gestionada por cookies HTTP-only con refresco automático.

2.2.2 Módulo de recetas

- Listado público de recetas con búsqueda por título e ingredientes.
- Vista de detalle con ingredientes, pasos, video incrustado y metadatos.
- Gestión CRUD completa desde el panel administrativo.
- Soporte para imagen principal, miniatura y video de YouTube.

2.2.3 Módulo social

- Sistema de likes con actualización optimista.
- Favoritos privados por usuario (solo cada usuario ve los suyos).
- Comentarios con soporte de respuestas anidadas (threaded).
- Actualización en tiempo real de comentarios mediante Supabase Realtime.
- Edición y eliminación de comentarios propios.

2.2.4 Módulo de blog

- Publicaciones cortas tipo post con imágenes múltiples y video opcional.
- Likes y comentarios independientes del sistema de recetas.
- Feed cronológico inverso.

2.2.5 Módulo administrativo

- Panel protegido accesible únicamente para usuarios con flag `is_admin = true`.
- Alta, edición y baja de recetas con subida de archivos multimedia.
- Gestión de mensajes de contacto con marcado de leído.
- Actualización de la foto del chef en la página Sobre mí.

3. Arquitectura de la aplicación

3.1 Patrón arquitectónico

Ingrediente 791 adopta una arquitectura híbrida basada en el modelo de Next.js App Router, que combina componentes renderizados en el servidor (React Server Components) con componentes interactivos del cliente (Client Components). Esta separación permite optimizar el rendimiento inicial de carga mientras se preserva la interactividad donde es necesaria.

La aplicación se apoya en Supabase como backend como servicio (BaaS), lo que elimina la necesidad de un servidor propio y delega en la plataforma la gestión de la base de datos, autenticación, almacenamiento de archivos y suscripciones en tiempo real.

3.2 Capas del sistema

El sistema se organiza en cuatro capas principales, que se comunican a través de interfaces bien definidas:



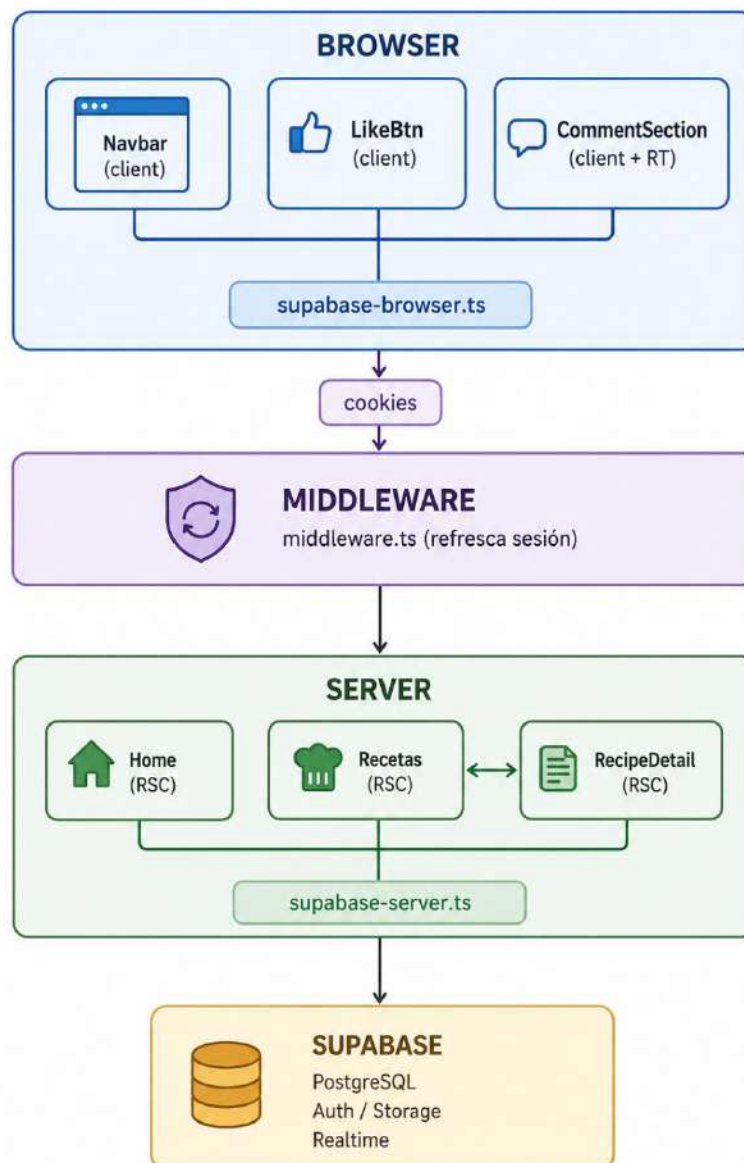
Arquitectura de la Aplicación

Capas de la arquitectura y sus responsabilidades.

Capa	Componentes	Responsabilidad
 Presentación	Componentes React (Server y Client)	Renderizado de la interfaz, gestión del estado local e interacción con el usuario.
 Middleware	<code>middleware.ts</code>	Refresco automático de la sesión y protección de rutas administrativas.
 Acceso a datos	<code>supabase-browser.ts</code> / <code>supabase-server.ts</code>	Abstracción sobre el cliente Supabase para browser y servidor.
 Persistencia	Supabase (PostgreSQL + Storage)	Almacenamiento de datos relacionales, archivos multimedia y eventos en tiempo real.

3.3 Diagrama de arquitectura

El siguiente diagrama representa el flujo de datos entre las distintas capas del sistema:



3.4 Regla de clientes Supabase

El proyecto establece una regla estricta para evitar errores comunes relacionados con la gestión de cookies y sesiones:



Estructura del Código

Organización de archivos por entorno y su propósito.

Entorno	Archivo	Uso
 Browser	<code>supabase-browser.ts</code>	Componentes con 'use client'
 Server	<code>supabase-server.ts</code>	Server Components y Route Handlers
 Middleware	<code>middleware.ts</code>	Refresco de sesión (inline)

IMPORTANTE: nunca se debe utilizar `createClient()` directamente en componentes. El uso incorrecto del cliente provoca pérdida de sesión y errores de autenticación difíciles de diagnosticar.

4. Stack tecnológico

A continuación se detalla el conjunto de tecnologías que conforman el sistema, con sus versiones específicas y la justificación de su elección.

4.1 Frontend



Tecnologías Utilizadas

Principales tecnologías y herramientas del stack del proyecto.

Tecnología	Versión	Función
 Next.js	14.2.21	Framework de React con App Router, Server Components y enrutamiento basado en archivos.
 React	18.3.1	Librería base para la construcción de componentes de interfaz.
 TypeScript	5.x	Tipado estático sobre JavaScript para mayor seguridad y autocompletado.
 Tailwind CSS	3.4.17	Framework de utilidades CSS para estilado rápido y consistente.
 Lucide React	0.460.0	Biblioteca de iconos SVG optimizados para React.

4.2 Backend como servicio (BaaS)



Tecnologías de Backend (Supabase)

Servicios y herramientas utilizadas en la infraestructura de Supabase.

Tecnología	Versión	Función
 @supabase/ssr	0.5.2	Cliente Supabase con soporte para Server Side Rendering y cookies.
 @supabase/supabase-js	2.47.10	SDK oficial para interactuar con Supabase desde JavaScript.
 PostgreSQL	15+	Motor de base de datos relacional provisto por Supabase.
 Supabase Auth	—	Servicio de autenticación con email, OAuth y recuperación de contraseña.
 Supabase Storage	—	Almacenamiento de archivos con buckets públicos y políticas de acceso.
 Supabase Realtime	—	Suscripción a cambios de base de datos en tiempo real vía WebSockets.

4.3 Servicios externos

- **Cloudflare Turnstile:** CAPTCHA invisible utilizado en el formulario de registro para prevenir bots.
- **Google OAuth:** proveedor de identidad federada alternativo al login por correo.
- **YouTube:** los videos de recetas se incrustan vía iframe desde enlaces de YouTube.

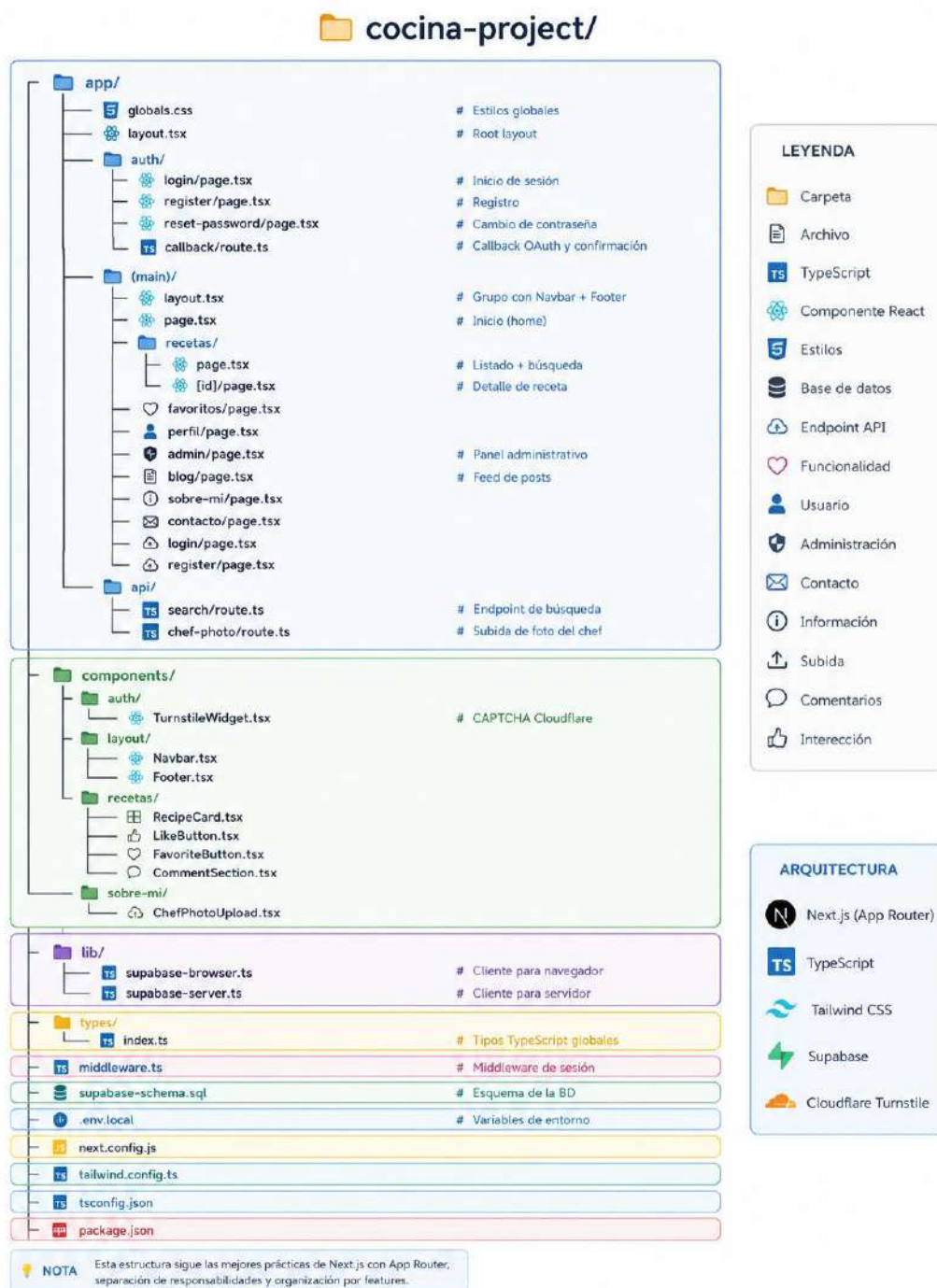
4.4 Requisitos de entorno

- **Node.js:** versión 18.17.0 o superior (requerido por Next.js 14).
- **npm:** versión 9 o superior.
- **Cuenta Supabase:** con proyecto activo y credenciales configuradas.
- **Navegador moderno:** Chrome, Firefox, Safari o Edge en sus versiones recientes.

5. Estructura del proyecto

5.1 Organización de carpetas

El proyecto sigue la convención de Next.js App Router, donde cada carpeta dentro de `app/` representa una ruta URL. Los componentes reutilizables se agrupan bajo `components/` y las utilidades bajo `lib/`.



5.2 Archivos clave

5.2.1 middleware.ts

Se ejecuta antes de cada request (excepto estáticos). Tiene dos responsabilidades: refrescar el token de sesión cuando está próximo a expirar, y proteger las rutas bajo /admin redirigiendo al login cuando el usuario no está autenticado.

5.2.2 lib/supabase-server.ts

Expone la función `createSupabaseServer()` que crea una instancia del cliente Supabase vinculada a las cookies del request actual. Se utiliza exclusivamente en Server Components y Route Handlers.

5.2.3 lib/supabase-browser.ts

Expone `createSupabaseBrowser()` que crea un cliente singleton para componentes marcados con 'use client'. Gestiona las cookies desde el lado del navegador.

5.2.4 types/index.ts

Centraliza los tipos TypeScript de las entidades del dominio: Recipe, UserProfile, Comment, Like, Favorite, Post y PostComment. Todos los componentes importan desde aquí para mantener consistencia.

6. Modelo de base de datos

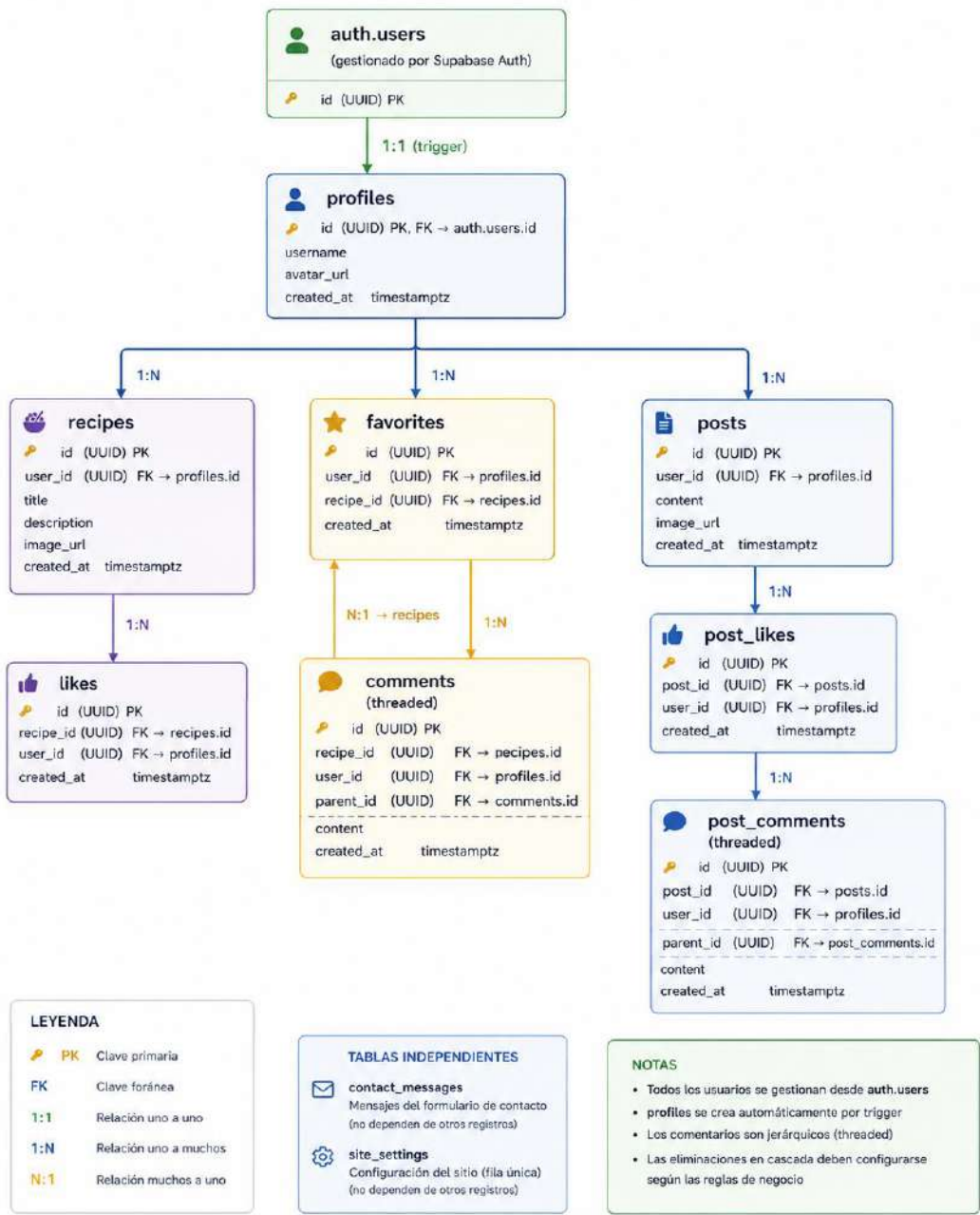
6.1 Visión general

La base de datos relacional del sistema se aloja en Supabase (PostgreSQL 15) y consta de doce tablas en el esquema public, más los buckets de Storage y la integración con el esquema auth. Todas las tablas implementan Row Level Security (RLS) para garantizar que el acceso a los datos respete las reglas de negocio.

El modelo se apoya fuertemente en el uso de UUIDs como claves primarias, generados automáticamente mediante la función `gen_random_uuid()`. Las relaciones entre tablas emplean `ON DELETE CASCADE` para mantener la integridad referencial: por ejemplo, al eliminar un usuario se eliminan en cascada sus likes, comentarios y favoritos.

6.2 Diagrama entidad-relación

Las relaciones entre las tablas principales son las siguientes:



6.3 Descripción detallada de las tablas

6.3.1 profiles

Perfil extendido de cada usuario. Se crea automáticamente mediante un trigger al registrarse un nuevo usuario en auth.users. Almacena información pública visible en la interfaz.



Tabla: profiles

Información del perfil de usuario (relacionada con auth.users)

Columna	Tipo	Null	Descripción
 id PK, FK	UUID (PK, FK → auth.users)	NO	Identificador único, coincide con el id de auth.users.
 email	TEXT	NO	Correo electrónico del usuario.
 full_name	TEXT	SÍ	Nombre completo ingresado en el registro.
 avatar_url	TEXT	SÍ	URL de la foto de perfil.
 chef_photo_url	TEXT	SÍ	URL de la foto del chef (solo admin).
 is_admin	BOOLEAN	NO	Bandera de rol administrador. Default: false.
 created_at	TIMESTAMPTZ	NO	Fecha de creación del registro.

Leyenda

 **PK** Clave primaria

 **FK** Clave foránea

 **NO** No permite valores nulos

 **SÍ** Permite valores nulos

 **Notas**

Esta tabla se crea generalmente mediante un trigger al registrarse un nuevo usuario en auth.users.

6.3.2 recipes

Tabla central del sistema. Contiene todas las recetas publicadas por los administradores, con sus ingredientes, pasos y recursos multimedia.



Tabla: recipes

Almacena las recetas publicadas en la plataforma.

Columna	Tipo	Null	Descripción
id	UUID (PK)	NO	Identificador único generado.
title	TEXT	NO	Título de la receta.
slug	TEXT UNIQUE	NO	Identificador amigable para URL.
description	TEXT	NO	Descripción breve mostrada en el listado.
ingredients	TEXT[]	NO	Array de ingredientes, uno por posición.
steps	TEXT[]	NO	Array de pasos de preparación.
prep_time	INTEGER	NO	Tiempo de preparación en minutos. Default: 30.
servings	INTEGER	NO	Número de porciones. Default: 4.
difficulty	TEXT (CHECK)	NO	Valores: 'fácil', 'media', 'difícil'.
image_url	TEXT	SÍ	URL de la imagen principal (1280x720).
thumbnail_url	TEXT	SÍ	URL de la miniatura (tarjeta).
video_url	TEXT	SÍ	URL de embed de YouTube.
author_id	UUID (FK → profiles.id)	NO	Autor de la receta.
created_at	TIMESTAMPTZ	NO	Fecha de publicación.
updated_at	TIMESTAMPTZ	NO	Fecha de última modificación.

Leyenda

PK Clave primaria
FK Clave foránea

SÍ Permite valores nulos
NO No permite valores nulos
[] Tipo array

Valores permitidos (difficulty)

'fácil' 'media' 'difícil'

Notas

- slug debe ser único para evitar URLs duplicadas.
- Los arrays (ingredients, steps) guardan un elemento por posición.

6.3.3 likes

Registro de los 'me gusta' dados por los usuarios a las recetas. La restricción UNIQUE(recipe_id, user_id) garantiza que un usuario no pueda dar más de un like a la misma receta.



Tabla: likes

Almacena los likes que los usuarios dan a las recetas.

Columna	Tipo	Descripción
 id	UUID (PK)	Identificador único del like.
 recipe_id	UUID (FK → recipes.id)	Receta a la que pertenece el like.
 user_id	UUID (FK → profiles.id)	Usuario que dio el like.
 created_at	TIMESTAMPTZ	Fecha en que se dio el like.

 PK

Clave primaria

 FK

Clave foránea

 TIMESTAMPTZ

Fecha y hora con zona horaria

 Notas

- Cada usuario puede dar like a una receta solo una vez.
- Para garantizarlo, se recomienda un índice único compuesto (user_id, recipe_id).
- El borrado en cascada debe configurarse en las relaciones con recipes y profiles.

6.3.4 comments

Comentarios realizados por los usuarios sobre las recetas. Soporta comentarios anidados mediante la columna `parent_id` que referencia al comentario padre.



Tabla: comments

Almacena los comentarios de las recetas con soporte para respuestas anidadas.

Columna	Tipo	Null	Descripción
 <code>id</code>	UUID (PK)	NO	Identificador único del comentario.
 <code>recipe_id</code>	UUID (FK → <code>recipes.id</code>)	NO	Receta comentada.
 <code>user_id</code>	UUID (FK → <code>profiles.id</code>)	NO	Autor del comentario.
 <code>parent_id</code>	UUID (FK → <code>comments.id</code>)	SÍ	Comentario padre (NULL para raíz).
 <code>content</code>	TEXT (CHECK <code>length > 0</code>)	NO	Contenido textual del comentario.
 <code>created_at</code>	TIMESTAMPTZ	NO	Fecha de publicación.

Leyenda

 **PK** Clave primaria

 **FK** Clave foránea

 **NO** No permite valores nulos

 **SÍ** Permite valores nulos

 **T** Texto

 Relación autorreferencial

 **Notas**

- Permite comentarios anidados usando `parent_id`.
- Para obtener hilos de comentarios, consulta recursivamente por `parent_id`.
- Se recomienda índice en `(recipe_id)` y `(parent_id)` para mejor rendimiento.

6.3.5 favorites

Recetas guardadas como favoritas por cada usuario. A diferencia de los likes, los favoritos son privados: solo el propio usuario puede ver los suyos (política RLS restrictiva).



Tabla: favorites

Almacena las recetas que los usuarios guardan como favoritas.

Columna	Tipo	Null	Descripción
 id	UUID (PK)	NO	Identificador único del favorito.
 recipe_id	UUID (FK → recipes.id)	NO	Receta guardada.
 user_id	UUID (FK → profiles.id)	NO	Usuario propietario del favorito.
 created_at	TIMESTAMPTZ	NO	Fecha de guardado.

Leyenda

 **PK** Clave primaria

 **FK** Clave foránea

 **TIMESTAMPTZ** Fecha y hora con zona horaria

NO No permite valores nulos

 **Notas**

- Cada usuario puede guardar una receta solo una vez.
- Para garantizarlo, se recomienda un índice único compuesto en `(user_id, recipe_id)`.

6.3.6 posts

Publicaciones cortas del blog. Son independientes de las recetas y permiten al administrador compartir contenido más informal con múltiples imágenes y video opcional.

Tabla: posts

Almacena las publicaciones (posts) del blog u otros contenidos.

Columna	Tipo	Null	Descripción
id	UUID (PK)	NO	Identificador único.
content	TEXT (CHECK length > 0)	NO	Texto de la publicación.
images	TEXT[]	NO	Array de URLs de imágenes adjuntas.
video_url	TEXT	SI	URL opcional de video.
author_id	UUID (FK → profiles.id)	NO	Autor de la publicación.
created_at	TIMESTAMPZ	NO	Fecha de publicación.
updated_at	TIMESTAMPZ	NO	Fecha de última modificación.

Leyenda

PK	Clave primaria	NO	No permite valores nulos
FK	Clave foránea	SI	Permite valores nulos
[]	Tipo array	TIMESTAMPZ	Fecha y hora con zona horaria

Notas

- El campo `content` debe tener al menos 1 carácter (CHECK length > 0).
- `images` es un array de URLs (ej: `["url1.jpg", "url2.png"]`).
- `video_url` es opcional y puede ser NULL.
- Se recomienda índices en `author_id` y `created_at` para rendimiento.

6.3.7 post_likes y post_comments

Tablas espejo de likes y comments pero asociadas a posts en lugar de recetas. Comparten la misma estructura y políticas de seguridad. La tabla post_comments también soporta respuestas anidadas mediante parent_id.

6.3.8 contact_messages

Mensajes enviados desde el formulario de contacto público. No requieren autenticación para su envío, pero solo los administradores pueden leerlos y marcarlos como leídos.


Tabla: contact_messages
 Almacena los mensajes enviados desde el formulario de contacto.

Columna	Tipo	Null	Descripción
 id	UUID (PK)	NO	Identificador único.
 name	TEXT	NO	Nombre del remitente.
 email	TEXT	NO	Correo del remitente.
 message	TEXT	NO	Cuerpo del mensaje.
 is_read	BOOLEAN	NO	Indica si el admin ya lo leyó. Default: false.
 created_at	TIMESTAMPZ	NO	Fecha de envío.

Leyenda

-  **PK** Clave primaria
- TEXT** Texto de longitud variable
- BOOLEAN** Valor verdadero o falso
- TIMESTAMPZ** Fecha y hora con zona horaria
- NO** No permite valores nulos

Notas

- Los mensajes llegan desde el formulario de contacto público.
- El campo `is_read` permite al admin marcar los mensajes como leídos.
- Se recomienda un índice en `(is_read, created_at)` para filtrar y ordenar eficientemente.

6.3.9 site_settings

Tabla de configuración global del sitio, con una única fila (id = 1). Actualmente almacena la URL de la foto del chef que se muestra en la página 'Sobre mí'. Diseñada para crecer con nuevos campos de configuración sin necesidad de crear tablas adicionales.

6.4 Índices

Se definen índices secundarios sobre las columnas más consultadas para acelerar operaciones de lectura:

- `idx_recipes_slug`: búsquedas de receta por slug.
- `idx_recipes_created`: ordenación cronológica inversa del listado.
- `idx_likes_recipe`, `idx_likes_user`: conteos y verificación de like por usuario.
- `idx_comments_recipe`, `idx_comments_parent`: carga de hilo de comentarios.
- `idx_favorites_user`: listado de favoritos de un usuario.
- `idx_posts_created`: feed del blog.
- `idx_post_likes_post`, `idx_post_comments_post`, `idx_post_comments_parent`.
- `idx_contact_created`: mensajes más recientes primero.

6.5 Triggers

El sistema define un trigger fundamental:



Trigger: `on_auth_user_created`

Se ejecuta automáticamente después de que se crea un nuevo usuario en la tabla `auth.users`.

</> SQL

```
CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();
```

La función `handle_new_user()` inserta automáticamente una fila en `profiles` cada vez que se crea un usuario en `auth.users`, copiando el id, email y los metadatos (nombre completo, avatar) provistos en el registro.



7. Políticas de seguridad (RLS)

7.1 Introducción a Row Level Security

Supabase utiliza PostgreSQL con Row Level Security habilitado en todas las tablas del esquema public. Esto significa que cada consulta SQL realizada desde el cliente es filtrada por el motor de base de datos aplicando las políticas (POLICY) definidas, independientemente de lo que envíe el código de la aplicación.

Este enfoque declarativo de la seguridad ofrece varias ventajas: las reglas de acceso se definen una única vez, a nivel de base de datos, y se aplican de manera uniforme sin importar el punto de entrada (web, API directa, panel de Supabase).

7.2 Matriz de políticas

La siguiente matriz resume las políticas aplicadas a cada tabla:



Matriz de permisos por tabla

Define quién puede realizar operaciones SELECT, INSERT, UPDATE y DELETE en cada tabla.

Tabla	SELECT	INSERT	UPDATE	DELETE	Descripción
 profiles	Público	Trigger	Propio	—	Perfiles de usuario. Se crea automáticamente via trigger.
 recipes	Público	Autenticado	Autenticado	Autenticado	Recetas publicadas por los usuarios.
 likes	Público	Propio	—	Propio	Likes en recetas (uno por usuario/receta).
 comments	Público	Propio	Propio	Propio	Comentarios en recetas y respuestas.
 favorites	Propio	Propio	—	Propio	Recetas guardadas por el usuario.
 posts	Público	Autenticado	Autenticado	Autenticado	Publicaciones del blog u otros contenidos.
 post_likes	Público	Propio	—	Propio	Likes en publicaciones del blog.
 post_comments	Público	Propio	Propio	Propio	Comentarios en publicaciones del blog.
 contact_messages	Autenticado	Anónimo	Autenticado	Autenticado	Mensajes enviados desde el formulario de contacto.

Roles

-  **Público**
Cualquier usuario, autenticado o no.
-  **Autenticado**
Usuarios con sesión iniciada.
-  **Anónimo**
Usuarios sin sesión iniciada.

-  **Propio**
Solo el propietario del registro.
-  **Trigger**
Gestionado automáticamente por trigger.
-  **—**
No aplica / No permitido.

Notas importantes

- Las políticas **RLS** (Row Level Security) aseguran que cada usuario solo pueda acceder a sus propios datos cuando corresponde (Propio).
- El acceso **Público** permite solo **SELECT**.
- INSERT, UPDATE y DELETE requieren autenticación.
- El trigger en **profiles** se ejecuta al crear un usuario en **auth.users**.

Recomendaciones

- ✓ Verifica siempre las políticas RLS en Supabase.
- ✓ Prueba los permisos con diferentes roles.
- ✓ Evita exponer datos sensibles en tablas públicas.

7.3 Ejemplos de políticas

7.3.1 Política de lectura pública

Permite que cualquier visitante, incluso sin autenticar, consulte todas las recetas:

**Política RLS**
Permite la lectura pública de recetas.

</> SQL

```
CREATE POLICY "Recipes: lectura pública"
ON public.recipes FOR SELECT
USING (true);
```

7.3.2 Política de propiedad

Un usuario solo puede insertar un like registrado a su propio nombre, evitando que dé likes en nombre de otros:

**Política RLS**
Permite que usuarios autenticados inserten likes únicamente para sí mismos.

</> SQL

```
CREATE POLICY "Likes: usuario autenticado inserta"
ON public.likes FOR INSERT
WITH CHECK (auth.uid() = user_id);
```

7.3.3 Política restrictiva (favoritos)

Los favoritos son privados. Ningún usuario puede ver los favoritos de otros:



7.4 Validación adicional en la aplicación

Aunque las políticas RLS proporcionan la seguridad fundamental, la aplicación implementa validaciones adicionales para mejorar la experiencia de usuario:

- Verificación del flag `is_admin` antes de mostrar el panel administrativo.
- Validación de tipo MIME y tamaño de archivos antes de subirlos a Storage.
- Sanitización de entradas textuales (longitud, caracteres especiales).
- Verificación del token de Cloudflare Turnstile en el registro.

8. Módulos funcionales

8.1 Módulo de autenticación

Implementa el flujo completo de alta, login, recuperación de contraseña y cierre de sesión. Se apoya en Supabase Auth y utiliza cookies HTTP-only para la persistencia.

8.1.1 Registro

El formulario de registro solicita nombre completo, correo electrónico y contraseña. Antes de enviar los datos, el usuario debe superar el CAPTCHA de Cloudflare Turnstile, cuyo token se valida del lado de Supabase. Tras un registro exitoso, el trigger `on_auth_user_created` crea automáticamente la fila correspondiente en la tabla `profiles`.

8.1.2 Login

Soporta dos métodos de autenticación: correo + contraseña, y Google OAuth. Ambos redirigen al usuario a la ruta indicada por `next` (o al inicio) tras la autenticación exitosa.

8.1.3 Recuperación de contraseña

El usuario ingresa su correo y recibe un enlace con token de recuperación. Al hacer click, se abre una página donde puede establecer una nueva contraseña. El sistema verifica que la sesión temporal (`PASSWORD_RECOVERY`) sea válida antes de permitir el cambio.

8.2 Módulo de recetas

8.2.1 Listado y búsqueda

La página `/recetas` muestra todas las recetas publicadas, ordenadas de la más reciente a la más antigua. Incluye un buscador que consulta por título e ingredientes, implementado en el endpoint `/api/search`.

8.2.2 Detalle de receta

La ruta `/recetas/[id]` renderiza una vista completa con imagen, video embebido, ingredientes en columna lateral y pasos numerados en el cuerpo principal. Los botones de like y favorito se cargan como componentes cliente con estado optimista.

8.2.3 CRUD administrativo

Desde `/admin`, el administrador puede crear, editar y eliminar recetas. El formulario valida que:

- Las imágenes sean exclusivamente en formato `.webp`.
- Los ingredientes y pasos se ingresen uno por línea.
- El slug sea único y URL-friendly.
- La dificultad esté entre los valores permitidos.

8.3 Módulo social

8.3.1 Sistema de likes

El componente `LikeButton` implementa el patrón de actualización optimista: al hacer click, se incrementa el contador y cambia el estado visual inmediatamente, y luego se envía la petición al servidor. Si la petición falla, se revierte el estado.

8.3.2 Sistema de favoritos

Similar a los likes pero privado. Los usuarios pueden listar sus favoritos en la ruta `/favoritos`. Al eliminar un favorito desde el botón del corazón, se elimina la fila correspondiente en la tabla `favoritos`.

8.3.3 Comentarios en tiempo real

El componente `CommentSection` se suscribe mediante Supabase Realtime al canal de la tabla `comments` filtrado por `recipe_id`. Cuando otro usuario publica un comentario, aparece instantáneamente en la pantalla sin necesidad de recargar la página.

Los comentarios soportan una profundidad de un nivel de respuesta (`parent_id` apunta al comentario raíz). El usuario puede editar o eliminar únicamente sus propios comentarios, lo que se garantiza tanto en la UI como a nivel de política RLS.

8.4 Módulo de blog

Equivalente simplificado del módulo de recetas, pensado para publicaciones cortas e informales. Admite hasta múltiples imágenes y un video opcional. Los posts tienen su propio sistema de likes y comentarios, completamente aislado del de las recetas.

8.5 Módulo administrativo

Ubicado en `/admin` y protegido por el middleware que redirige al login si el usuario no está autenticado. Una verificación adicional en el componente asegura que el usuario tenga el flag `is_admin = true`, mostrando un mensaje de error en caso contrario.

Funciones disponibles en el panel:

- Alta, edición y baja de recetas.
- Subida de imagen principal, miniatura y video.
- Gestión de posts del blog.
- Bandeja de mensajes de contacto con marcado de leído.
- Actualización de la foto del chef.

9. Gestión de archivos (Storage)

9.1 Buckets definidos

Supabase Storage organiza los archivos en buckets. El sistema define tres buckets públicos:

 Buckets de Storage Buckets utilizados para almacenar archivos e imágenes del sistema.		
Bucket	Visibilidad	Contenido
 recipe-images	Público	Imágenes principales y miniaturas de recetas. También la foto del chef en algunos casos.
 recipe-videos	Público	Videos asociados a recetas (uso opcional: normalmente se usa YouTube embed).
 post-images	Público	Imágenes adjuntas a publicaciones del blog.
 site-assets	Público	Recursos globales del sitio (foto del chef en Sobre mí).

9.2 Políticas de acceso a Storage

Cada bucket tiene sus propias políticas, separadas de las políticas de tablas:

- **Lectura pública:** cualquier visitante puede ver las URLs. No es necesario estar autenticado.
- **Escritura restringida:** solo usuarios autenticados pueden subir archivos. La validación adicional de rol admin se realiza en el código de la aplicación.

9.3 Convenciones de nombres

Para evitar colisiones, los archivos se nombran siguiendo patrones predecibles:

- Imágenes de recetas: {slug}-{timestamp}.webp
- Foto del chef: chef-photo-{timestamp}.webp
- Imágenes de posts: post-{timestamp}-{index}.webp

9.4 Formato obligatorio: WebP

El sistema impone el uso exclusivo de imágenes en formato WebP para reducir el peso de carga y mejorar el rendimiento. El validador del formulario rechaza cualquier otro formato.

9.5 Límites de tamaño

- Imágenes: máximo 5 MB por archivo.
- Videos: máximo 50 MB (poco utilizado, se prefiere YouTube).
- Imágenes recomendadas: 1280 x 720 píxeles para imagen principal, 600 x 400 para miniatura.

10. Autenticación y middleware

10.1 Flujo de sesión

Supabase gestiona la sesión mediante cookies HTTP-only que contienen el access token y el refresh token. El cliente de la aplicación nunca accede directamente a estos tokens: los gestiona el paquete `@supabase/ssr`.

El ciclo de vida de una sesión es el siguiente:

1. El usuario hace login y Supabase emite un par de tokens.
2. Las cookies se establecen en el navegador con flag `HttpOnly` y `Secure`.
3. En cada request, el `middleware.ts` intercepta la petición, lee las cookies y refresca el access token si está próximo a expirar.
4. Las cookies actualizadas se propagan tanto al request (para Server Components downstream) como al response (para el navegador).
5. Al cerrar sesión, las cookies se eliminan y se revocan los tokens en el servidor.

10.2 Código del middleware

El middleware es el corazón del sistema de autenticación. Sin él, las sesiones expiran silenciosamente y el usuario debe volver a iniciar sesión con frecuencia.

```
import { createServerClient } from '@supabase/ssr';
import { NextResponse, type NextRequest } from 'next/server';

export async function middleware(request: NextRequest) {
  let supabaseResponse = NextResponse.next({ request });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) =>
            request.cookies.set(name, value)
          );
          supabaseResponse = NextResponse.next({ request });
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options)
          );
        },
      },
    },
  );

  // CRÍTICO: refresca el token si está por expirar
  const { data: { user } } = await supabase.auth.getUser();

  // Proteger rutas admin
  if (request.nextUrl.pathname.startsWith('/admin') && !user) {
    const url = request.nextUrl.clone();
    url.pathname = '/login';
    return NextResponse.redirect(url);
  }
  return supabaseResponse;
}

export const config = {
  matcher: [
    '/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)',
  ],
};
```

10.3 Protección CAPTCHA

El formulario de registro incorpora Cloudflare Turnstile, un CAPTCHA invisible que verifica que el usuario sea humano sin necesidad de resolver imágenes o texto.

El flujo es el siguiente:

1. El componente TurnstileWidget renderiza el widget de Cloudflare.
2. Al completar la verificación, Cloudflare devuelve un token único.
3. Este token se envía junto con los datos de registro al endpoint `signUp()` de Supabase.
4. Supabase valida el token con Cloudflare antes de crear el usuario.
5. Si el token es inválido o ha sido reutilizado, el registro es rechazado.

11. Sistema de autenticación

11.1 Flujo general

La autenticación está construida sobre Supabase Auth, que maneja las contraseñas, los tokens JWT y las cookies. El flujo soporta dos métodos: correo y contraseña (con captcha de Cloudflare Turnstile) y Google OAuth.

Pasos del registro con correo

1. El usuario llena el formulario en `/auth/register`.
2. Se valida que la contraseña tenga mínimo 6 caracteres y que el captcha esté resuelto.
3. Se llama a `supabase.auth.signUp()` con el `captchaToken`.
4. Supabase crea el usuario en `auth.users`.
5. El trigger `handle_new_user` crea automáticamente el registro en `public.profiles`.
6. Supabase envía correo de confirmación (si está activado).
7. El usuario hace click en el enlace → redirige a `/auth/callback?code=xxx`.
8. El route handler intercambia el code por una sesión y redirige a la página principal.

11.2 Cliente de servidor: supabase-server.ts

Este cliente se usa en Server Components, Route Handlers y Server Actions. Lee las cookies del request y crea una instancia stateless por cada petición.

```
1  // lib/supabase-server.ts
2  import { createServerClient } from '@supabase/ssr';
3  import { cookies } from 'next/headers';
4
5  export async function createSupabaseServer() {
6    const cookieStore = await cookies();
7
8    return createServerClient(
9      process.env.NEXT_PUBLIC_SUPABASE_URL!,
10     process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
11     {
12       cookies: {
13         getAll() {
14           return cookieStore.getAll();
15         },
16         setAll(cookiesToSet) {
17           try {
18             cookiesToSet.forEach(({ name, value, options }) => {
19               cookieStore.set(name, value, options);
20             });
21           } catch {
22             // setAll falla en Server Components (read-only)
23             // El middleware se encarga de refrescar.
24           }
25         },
26       },
27     },
28   );
29 }
```


11.3 Cliente de navegador: supabase-browser.ts

Cliente ligero para componentes 'use client'. Se usa en LikeButton, FavoriteButton, CommentSection y todos los formularios interactivos.

```
1  // lib/supabase-browser.ts
2  import { createBrowserClient } from '@supabase/ssr';
3
4  export function createSupabaseBrowser() {
5    return createBrowserClient(
6      process.env.NEXT_PUBLIC_SUPABASE_URL!,
7      process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
8    );
9  }
```

11.4 Middleware: refresco automático de sesión

El middleware se ejecuta en cada request (excepto archivos estáticos). Su función crítica es llamar a `supabase.auth.getUser()`, que refresca el token JWT si está próximo a expirar. Sin este middleware, las sesiones expirarían silenciosamente y los usuarios serían desconectados de manera errática. Además, el middleware protege las rutas administrativas: si un usuario no autenticado intenta acceder a cualquier ruta bajo `/admin`, es redirigido a `/login`.

```
1 // middleware.ts
2 import { createServerClient } from '@supabase/ssr';
3 import { NextResponse, type NextRequest } from 'next/server';
4
5 export async function middleware(request: NextRequest) {
6   let supabaseResponse = NextResponse.next({ request });
7
8   const supabase = createServerClient(
9     process.env.NEXT_PUBLIC_SUPABASE_URL!,
10    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
11    {
12      cookies: {
13        getAll() { return request.cookies.getAll(); },
14        setAll(cookiesToSet) {
15          cookiesToSet.forEach(({ name, value }) => {
16            request.cookies.set(name, value);
17          });
18          supabaseResponse = NextResponse.next({ request });
19          cookiesToSet.forEach(({ name, value, options }) => {
20            supabaseResponse.cookies.set(name, value, options);
21          });
22        },
23      },
24    }
25  );
26
27  // IMPORTANTE: refresca el token si está por expirar
28  const { data: { user } } = await supabase.auth.getUser();
29
30  // Proteger rutas admin
31  if (request.nextUrl.pathname.startsWith('/admin') && !user) {
32    const url = request.nextUrl.clone();
33    url.pathname = '/login';
34    return NextResponse.redirect(url);
35  }
36
37  return supabaseResponse;
38 }
39
40 export const config = {
41   matcher: [
42     '/(?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*',
43   ],
44 };
```

11.5 Callback de email: app/auth/callback/route.ts

Route Handler que procesa el código enviado por Supabase tras la confirmación de email o la solicitud de recuperación de contraseña. Convierte el code en una sesión activa.

```
1 // app/auth/callback/route.ts
2 import { createSupabaseServer } from '@lib/supabase-server';
3 import { NextResponse } from 'next/server';
4
5 export async function GET(request: NextRequest) {
6   const { searchParams, origin } = new URL(request.url);
7   const code = searchParams.get('code');
8   const type = searchParams.get('type');
9
10  if (code) {
11    const supabase = await createSupabaseServer();
12    await supabase.auth.exchangeCodeForSession(code);
13
14    if (type === 'recovery') {
15      return NextResponse.redirect(`${origin}/auth/reset-password`);
16    }
17  }
18  return NextResponse.redirect(`${origin}/`);
19 }
20 }
```

11.6 Verificación de admin

Para verificar si un usuario es administrador, todas las rutas protegidas siguen el mismo patrón: obtener el usuario, consultar su perfil y verificar el campo `is_admin`.

```
1 // app/(main)/admin/page.tsx
2 const supabase = await createSupabaseServer();
3 const { data: { user } } = await supabase.auth.getUser();
4
5 if (!user) redirect('/login');
6
7 const { data: profile } = await supabase
8   .from('profiles')
9   .select('is_admin')
10   .eq('id', user.id)
11   .single();
12 if (!profile?.is_admin) redirect('/');
```

12. Seguridad

12.1 Capas de defensa

El sistema implementa seguridad en múltiples capas, asumiendo que cualquier una puede fallar:

1. Middleware: redirige a /login si no hay sesión en rutas protegidas.
2. Server Component: verifica `is_admin` antes de renderizar /admin.
3. Row Level Security (RLS): aunque alguien forme una petición manual, PostgreSQL rechaza la operación.
4. Políticas de Storage: solo usuarios autenticados pueden subir, lectura pública.
5. Captcha de Cloudflare Turnstile en el registro para evitar bots.
6. Validación de tipo de archivo en cliente y en políticas de bucket.

12.2 Row Level Security (RLS)

RLS es la línea de defensa más importante. Cada tabla tiene políticas que definen qué puede hacer cada usuario. La regla es: si no hay política que permita la operación, está prohibida.

Ejemplos de políticas críticas

```
1  -- Solo el admin puede modificar recetas
2  CREATE POLICY "Recipes: admin inserta"
3    ON public.recipes FOR INSERT
4    WITH CHECK (
5      EXISTS (SELECT 1 FROM public.profiles
6              WHERE id = auth.uid() AND is_admin = true)
7    );
8
9  -- Solo el usuario ve sus propios favoritos
10 CREATE POLICY "Favorites: usuario ve sus favoritos"
11   ON public.favorites FOR SELECT
12   USING (auth.uid() = user_id);
13
14 -- Solo el usuario puede borrar SU comentario
15 CREATE POLICY "Comments: usuario elimina su comentario"
16   ON public.comments FOR DELETE
17   USING (auth.uid() = user_id);
18
19 -- Solo el usuario puede editar SU comentario
20 CREATE POLICY "Comments: usuario actualiza su comentario"
21   ON public.comments FOR UPDATE
22   USING (auth.uid() = user_id)
23   WITH CHECK (auth.uid() = user_id);
```

12.3 Cookies seguras

Las cookies de sesión de Supabase son HTTP-only (no accesibles desde JavaScript), lo que las protege contra ataques XSS. El middleware se encarga de renovarlas automáticamente.

12.4 Variables de entorno

Anon key: es pública por diseño. Su seguridad reside en las políticas RLS de la base de datos.

Service role key: NO se usa en este proyecto. Si en algún momento se agrega, NUNCA debe usarse en cliente ni con prefijo NEXT_PUBLIC_, porque salta todas las políticas RLS.

12.5 Captcha en registro

El formulario de registro integra Cloudflare Turnstile. El captchaToken se valida del lado de Supabase, no del cliente, lo que impide bypass por manipulación del frontend.

```
1  const { error } = await supabase.auth.signUp({
2    email: emailCheck.email,
3    password,
4    options: {
5      data: { full_name: fullName.trim() },
6      emailRedirectTo: `${window.location.origin}/auth/callback`,
7      captchaToken, // Supabase lo verifica con Cloudflare
8    },
9  });
10
```

13. Sistema de búsqueda

13.1 Arquitectura del buscador

El sistema tiene tres buscadores complementarios, cada uno con responsabilidades específicas:

Buscador	Descripción
Buscador del Navbar	Buscador en vivo, desktop + móvil, con dropdown de sugerencias. Está integrado dentro de Navbar.tsx.
LiveSearchBar (página /recetas)	Versión reutilizable del buscador, usado en la página de listado. Comparte caché con el navbar.
Buscador del Admin Panel	Filtro local sobre las recetas ya cargadas. Usa scoring (puntuación) por título e ingredientes.

13.2 API de búsqueda: /api/search/route.ts

Endpoint GET que devuelve sugerencias rápidas para el dropdown. Sin query, devuelve las cinco recetas más populares (más likes). Con query, busca en título e ingredientes.

Lógica de búsqueda

La búsqueda hace dos consultas a Supabase porque el operador 'cs' (contains) solo busca match exacto del elemento del array, lo que no sirve para buscar 'huevo' dentro de '3 huevos grandes'. La solución es traer un conjunto amplio y filtrar en JavaScript.


```

1  // TYPESCRIPT
2  // app/api/search/route.ts
3  import { createSupabaseServer } from '@lib/supabase-server';
4  import { NextResponse } from 'next/server';
5
6  // GET /api/search?q=pollo -> buscar recetas (rankeadas)
7  // GET /api/search?q=      -> recetas populares (más likes)
8  export async function GET(request: Request) {
9    const { searchParams } = new URL(request.url);
10   const q = (searchParams.get('q') ?? '').trim();
11   const supabase = await createSupabaseServer();
12
13   // Sin query -> devolver populares
14   if (!q) {
15     const { data: popular } = await supabase
16       .from('recipes')
17       .select('id, title, slug, thumbnail_url, image_url,
18             difficulty, prep_time,
19             likes_count:likes(count)')
20       .order('created_at', { ascending: false })
21       .limit(20);
22
23     const sorted = (popular ?? []).
24       .map((r) => ({ ...r, likes_count: r.likes_count?.[0]?.count ?? 0 }));
25     sorted.sort((a, b) => b.likes_count - a.likes_count);
26     sorted.slice(0, 5);
27     return NextResponse.json({ recipes: sorted, type: 'popular' });
28   }
29
30   // Con query -> normalizar (sin acentos, minúsculas)
31   const qLower = q.toLowerCase();
32   const qNorm = qLower.normalize('NFD').replace(/[\u0300-\u036f]/g, '');
33
34   // 1) Buscar por título con ilike
35   const { data: byTitle } = await supabase
36     .from('recipes')
37     .select('id, title, slug, description, thumbnail_url,
38           image_url, difficulty, prep_time, ingredients,
39           likes_count:likes(count)')
40     .ilike('title', `%${q}%`)
41     .limit(10);
42
43   // 2) Traer set amplio y filtrar ingredientes en JS
44   const titleIds = new Set((byTitle ?? []).map(r => r.id));
45   const { data: allRecipes } = await supabase
46     .from('recipes')
47     .select('...')
48     .limit(100);
49
50   const byIngredients = (allRecipes ?? []).filter((r) => {
51     if (titleIds.has(r.id)) return false;
52     if (!Array.isArray(r.ingredients)) return false;
53     return r.ingredients.some((ing) => {
54       const ingLower = ing.toLowerCase();
55       const ingNorm = ingLower.normalize('NFD').replace(/[\u0300-\u036f]/g, '');
56       return ingLower.includes(qLower) || ingNorm.includes(qNorm);
57     });
58   });
59   // Combinar resultados...
60   return NextResponse.json({ recipes: combined, type: 'results' });
61 }

```

13.3 Componente LiveSearchBar

Componente cliente reutilizable que implementa el buscador con dropdown. Combina varios patrones avanzados de React para una experiencia fluida.

Características técnicas clave

- **Debounce de 300ms:** evita disparar una petición por cada tecla. Espera a que el usuario termine de escribir.
- **AbortController:** cancela las peticiones obsoletas si el usuario sigue escribiendo. Solo el último fetch se aplica.
- **Caché en memoria (60s TTL):** guarda resultados por término de búsqueda. Compartido entre todas las instancias de LiveSearchBar.
- **Navegación por teclado:** flechas ↑ ↓ para moverse en las sugerencias, Enter para seleccionar, Escape para cerrar.
- **Sync con URL:** si la URL trae ?search=xxx, el input se rellena automáticamente.
- **Cleanup en unmount:** limpia timeouts y aborta fetches pendientes para evitar memory leaks.

Hook principal: fetchSuggestions

```
1 // TYPESCRIPT
2 // components/search/LiveSearchBar.tsx
3 const fetchSuggestions = useCallback(async (q: string) => {
4   // 1) Revisar caché
5   const cacheKey = q.toLowerCase().trim();
6   const cached = getCached(cacheKey);
7   if (cached) {
8     setSuggestions(cached.recipes);
9     setSuggestionsType(cached.type);
10    setSearching(false);
11    return;
12  }
13
14  // 2) Abortar fetch anterior si existe
15  if (abortRef.current) abortRef.current.abort();
16  const controller = new AbortController();
17  abortRef.current = controller;
18
19  // 3) Hacer fetch
20  setSearching(true);
21  try {
22    const res = await fetch(
23      `/api/search?q=${encodeURIComponent(q)}`,
24      { signal: controller.signal }
25    );
26    const data = await res.json();
27    const recipes = data.recipes ?? [];
28    const type = data.type ?? 'empty';
29
30    setCache(cacheKey, recipes, type);
31    if (!controller.signal.aborted) {
32      setSuggestions(recipes);
33      setSuggestionsType(type);
34    }
35  } catch (err: any) {
36    if (err.name !== 'AbortError') {
37      setSuggestions([]);
38      setSuggestionsType('empty');
39    }
40  } finally {
41    if (!controller.signal.aborted) setSearching(false);
42  }
43 }, []);
44
46
```

Handler de cambio de input con debounce

```
1  // TYPESCRIPT
2  // utils/dedup.ts
3  export function uniqueBy<T>(array: T[], key: (item: T) => string): T[] {
4      const seen = new Set<string>();
5      return array.filter(item => {
6          const k = key(item);
7          if (seen.has(k)) return false;
8          seen.add(k);
9          return true;
10     });
11 }
```

Navegación por teclado

```
1 // TYPESCRIPT
2 // components/search/LiveSearchBar.tsx
3 const handleKeyDown = (e: React.KeyboardEvent) => {
4   if (e.key === 'Escape') {
5     setShowDropdown(false);
6     inputRef.current?.blur();
7     return;
8   }
9   if (!showDropdown || suggestions.length === 0) return;
10
11   switch (e.key) {
12     case 'ArrowDown':
13       e.preventDefault();
14       setActiveIndex((prev) =>
15         prev < suggestions.length - 1 ? prev + 1 : 0
16       );
17       break;
18     case 'ArrowUp':
19       e.preventDefault();
20       setActiveIndex((prev) =>
21         prev > 0 ? prev - 1 : suggestions.length - 1
22       );
23       break;
24     case 'Enter':
25       e.preventDefault();
26       if (activeIndex >= 0 && activeIndex < suggestions.length) {
27         navigateToRecipe(suggestions[activeIndex].slug);
28       } else if (query.trim()) {
29         navigateToSearch(query.trim());
30       }
31       break;
32   }
33 };
34
```

13.4 Caché de búsqueda: SearchDropdown.tsx

El caché está exportado desde SearchDropdown.tsx como un Map global. Esto significa que es compartido entre el buscador del Navbar y el LiveSearchBar de la página de recetas: si el usuario busca 'pollo' en el navbar y luego abre /recetas, el caché ya tiene los resultados.

```
1 // TYPESCRIPT
2 // components/search/SearchDropdown.tsx
3 const searchCache = new Map<string, {
4   recipes: SearchResult[];
5   type: string;
6   ts: number;
7 }>();
8 const CACHE_TTL = 60_000; // 60 segundos
9
10 export function getCached(key: string) {
11   const entry = searchCache.get(key);
12   if (!entry) return null;
13   if (Date.now() - entry.ts > CACHE_TTL) {
14     searchCache.delete(key);
15     return null;
16   }
17   return entry;
18 }
19
20 export function setCache(key: string, recipes: SearchResult[], type: string) {
21   // LRU básico: si pasa de 50 entradas, borrar la más antigua
22   if (searchCache.size > 50) {
23     const oldest = searchCache.keys().next().value;
24     if (oldest) searchCache.delete(oldest);
25   }
26   searchCache.set(key, { recipes, type, ts: Date.now() });
```


13.5 Buscador de la página /recetas (Server Component)

Cuando el usuario envía el formulario, la URL queda como /recetas?search=pollo. El Server Component lee el query, hace la consulta a Supabase y renderiza el HTML con los resultados ya listos. No depende del JavaScript del cliente para mostrar los resultados.

```
1  // TYPESCRIPT
2  // app/(main)/recetas/page.tsx
3  export default async function RecetasPage({ searchParams }) {
4    const params = await searchParams;
5    // Acepta ?search= y ?q= por compatibilidad
6    const query = (params.search ?? params.q ?? '').trim();
7    const supabase = await createSupabaseServer();
8
9    if (query) {
10     // Normalizar (sin acentos, minúsculas)
11     const qLower = query.toLowerCase();
12     const qNorm = qLower.normalize('NFD').replace(/[\u0300-\u036f]/g, '');
13
14     // Traer todas y filtrar en JS (igual que la API)
15     const { data } = await supabase
16       .from('recipes')
17       .select('*', likes_count:likes(count), comments_count:comments(count)')
18       .order('created_at', { ascending: false });
19
20     const filtered = (data ?? []).filter((r) => {
21       const titleLower = (r.title ?? '').toLowerCase();
22       const titleNorm = titleLower.normalize('NFD')
23         .replace(/[\u0300-\u036f]/g, '');
24
25       if (titleLower.includes(qLower) ||
26         titleNorm.includes(qNorm)) return true;
27
28       if (Array.isArray(r.ingredients)) {
29         return r.ingredients.some((ing) => {
30           const ingLower = ing.toLowerCase();
31           const ingNorm = ingLower.normalize('NFD')
32             .replace(/[\u0300-\u036f]/g, '');
33           return ingLower.includes(qLower) ||
34             ingNorm.includes(qNorm);
35         });
36       }
37
38       return false;
39     });
40     // ...
41   }
42   // ...
43 }
44 }
```

14. Componentes principales

14.1 RecipeCard

Tarjeta de presentación de una receta usada en el home, en /recetas, en favoritos y en cualquier listado. Es un componente puramente presentacional, no maneja estado.

Props

TYPESCRIPT

```
interface Props {  
  recipe: Recipe; // ver types/index.ts  
}
```

Características

- Renderiza imagen optimizada con next/image (lazy load + blur placeholder).
- Usa thumbnail_url si existe, sino image_url, sino un ícono de ChefHat.
- Muestra dificultad con código de colores: herb (fácil), honey (media), wine (difícil).
- Muestra prep_time, likes_count y comments_count.
- Toda la tarjeta es un Link a /recetas/[slug].

14.2 LikeButton — Optimistic Updates

Componente cliente que implementa el patrón de optimistic update: la UI se actualiza inmediatamente al hacer click, sin esperar respuesta del servidor. Si la operación falla, se revierte automáticamente.

```
1 // TYPESCRIPT
2 // components/recetas/LikeButton.tsx
3 'use client';
4
5 export default function LikeButton({
6   recipeId, userId, initialLiked, initialCount
7 }: Props) {
8   const [liked, setLiked] = useState(initialLiked);
9   const [count, setCount] = useState(initialCount);
10  const [loading, setLoading] = useState(false);
11  const router = useRouter();
12  const supabase = createSupabaseBrowser();
13
14  const handleToggle = async () => {
15    if (loading) return;
16
17    // Si no hay usuario -> redirigir a login
18    if (!userId) {
19      router.push('/login');
20      return;
21    }
22
23    // Guardar estado anterior por si hay que revertir
24    const prevLiked = liked;
25    const prevCount = count;
26
27    // ← Optimistic update: actualizar UI YA
28    setLiked(!prevLiked);
29    setCount(prevLiked ? prevCount - 1 : prevCount + 1);
30    setLoading(true);
31
32    try {
33      if (prevLiked) {
34        // Quitar like
35        const { error } = await supabase
36          .from('likes')
37          .delete()
38          .eq('recipe_id', recipeId)
39          .eq('user_id', userId);
40        if (error) throw error;
41      } else {
42        // Dar like
43        const { error } = await supabase
44          .from('likes')
45          .insert({ recipe_id: recipeId, user_id: userId });
46        if (error) throw error;
47      }
48      // Refrescar el Server Component
49      router.refresh();
50    } catch {
51      // ← Revertir si falló
52      setLiked(prevLiked);
53      setCount(prevCount);
54    } finally {
55      setLoading(false);
56    }
57  };
58
59  return (
60    <button onClick={handleToggle} disabled={loading} ...>
61      <Heart className={liked ? 'fill-wine' : ''} />
62      <span>{count}</span>
63    </button>
64  );
65 }
```

14.3 FavoriteButton

Funciona exactamente igual que LikeButton pero opera sobre la tabla favorites en lugar de likes. La diferencia clave: favorites tiene RLS que solo permite ver los propios favoritos del usuario.

14.4 CommentSection — Comentarios en tiempo real

Componente cliente complejo que maneja: comentarios planos y anidados (respuestas), suscripción Realtime a inserciones, edición y eliminación, autenticación reactiva.

Suscripción Realtime

```
1 // TYPESCRIPT
2 // components/recetas/CommentSection.tsx
3 useEffect(() => {
4   const channel = supabase
5     .channel(`comments:${recipeId}`)
6     .on(
7       'postgres_changes',
8       {
9         event: 'INSERT',
10        schema: 'public',
11        table: 'comments',
12        filter: `recipe_id=eq.${recipeId}`
13      },
14      async (payload) => {
15        // Cargar perfil del autor
16        const { data: profile } = await supabase
17          .from('profiles')
18          .select('*')
19          .eq('id', payload.new.user_id).single();
20
21        const newC: Comment = {
22          ...payload.new as Comment,
23          user: profile ?? undefined
24        };
25
26        // Evitar duplicados si el comentario es nuestro
27        setComments((prev) => {
28          if (prev.some((c) => c.id === newC.id)) return prev;
29          return [...prev, newC];
30        });
31      }
32    )
33    .subscribe();
34   return () => { supabase.removeChannel(channel); };
35 }, [recipeId]);
```

Envío de comentario o edición

```
1 // TYPESCRIPT
2 const handleSubmit = async (e: React.FormEvent) => {
3   e.preventDefault();
4   if (!newComment.trim() || !user) return;
5   setSending(true);
6
7   if (editingComment) {
8     // Modo edición
9     const { error } = await supabase
10       .from('comments')
11       .update({ content: newComment.trim() })
12       .eq('id', editingComment.id);
13
14     if (!error) {
15       setComments((prev) => prev.map((c) =>
16         c.id === editingComment.id
17           ? { ...c, content: newComment.trim() }
18           : c
19       ));
20     }
21     setEditingComment(null);
22   } else {
23     // Nuevo comentario o respuesta
24     const { error } = await supabase
25       .from('comments')
26       .insert({
27         recipe_id: recipeId,
28         user_id: user.id,
29         content: newComment.trim(),
30         parent_id: replyingTo?.id ?? null,
31       });
32   }
33   setNewComment('');
34   setReplyingTo(null);
35   setSending(false);
36 };
```

14.5 Navbar

Componente cliente que combina cuatro responsabilidades: navegación principal, estado de autenticación reactivo, buscador con dropdown (desktop y móvil), menú de usuario con logout.

Suscripción al estado de autenticación

```
1  // TYPESCRIPT
2  useEffect(() => {
3    // Listener para cambios de auth (login, logout)
4    const { data: { subscription } } = supabase.auth
5      .onAuthStateChange((_event, session) => {
6        setUser(session?.user ?? null);
7        // Recargar perfil para saber si es admin
8        if (session?.user) loadProfile(session.user.id);
9        else setIsAdmin(false);
10   });
11
12   // Carga inicial
13   loadInitialUser();
14
15   return () => { subscription.unsubscribe(); };
16 }, []);
17
```

14.6 AdminPanel

Componente cliente principal del panel administrativo. Recibe datos iniciales desde el Server Component padre y mantiene el estado de toda la UI. Tiene cuatro pestañas: Resumen, Recetas, Mensajes y Posts.

Estructura del estado

```
1  const [activeTab, setActiveTab] = useState<Tab>('overview');
2  const [recipes, setRecipes] = useState<Recipe[]>(initialRecipes);
3  const [messages, setMessages] = useState<ContactMessage[]>(initialMessages);
4  const [posts, setPosts] = useState<Post[]>(initialPosts);
5
6  // Estado del formulario de receta
7  const [form, setForm] = useState<RecipeForm>(emptyForm);
8  const [editingId, setEditingId] = useState<string | null>(null);
9  const [imageFile, setImageFile] = useState<File | null>(null);
10 const [thumbnailFile, setThumbnailFile] = useState<File | null>(null);
11
12 // Filtros
13 const [recipeSearch, setRecipeSearch] = useState('');
14 const [messageFilter, setMessageFilter] =
    useState<'all' | 'unread' | 'read'>('all');
```

15. Funciones utilitarias clave

15.1 slugify — Generar slug de URL

Convierte un título de receta en una cadena apta para URL: minúsculas, sin acentos, sin caracteres especiales, espacios convertidos a guiones. Es la base del enrutamiento de recetas.

```
1 // app/(main)/admin/AdminPanel.tsx
2 const slugify = (text: string) =>
3   text
4     .toLowerCase()
5     .normalize('NFD')
6     .replace(/[\u0300-\u036f]/g, '') // quitar acentos
7     .replace(/[^a-z0-9]/g, '-') // cualquier no-alfanumérico → guion
8     .replace(/(^-|-$)/g, ''); // quitar guiones de inicio/fin
9
10 // Ejemplos:
11 // "Tacos al Pastor" → "tacos-al-pastor"
12 // "¡Cebiche Peruano!" → "cebiche-peruano"
13 // "Crème Brûlée" → "creme-brulee"
14
15
16
```


15.2 extractYoutubeId — Extraer ID de YouTube

Detecta el ID de video en URLs de YouTube de cualquier formato común. Devuelve el ID o null si no es una URL válida. Es la base para embeber videos y para extraer miniaturas automáticamente.

```
1 // app/(main)/admin/AdminPanel.tsx
2 const extractYoutubeId = (url: string): string | null => {
3   if (!url.trim()) return null;
4
5   const patterns = [
6     /(?:youtube\\.com\\/watch\\?v=)([a-zA-Z0-9_-]{11})/,
7     /(?:youtu\\.be\\/)([a-zA-Z0-9_-]{11})/,
8     /(?:youtube\\.com\\/embed\\/)([a-zA-Z0-9_-]{11})/,
9     /(?:youtube\\.com\\/shorts\\/)([a-zA-Z0-9_-]{11})/,
10  ];
11
12  for (const pattern of patterns) {
13    const match = url.match(pattern);
14    if (match) return match[1];
15  }
16
17  return null;
18 };
19
20 // Ejemplos:
21 // "https://www.youtube.com/watch?v=dQw4w9WgXcQ" -> "dQw4w9WgXcQ"
22 // "https://youtu.be/dQw4w9WgXcQ" -> "dQw4w9WgXcQ"
23 // "https://youtube.com/shorts/abc123XYZ45" -> "abc123XYZ45"
24 // "https://vimeo.com/12345" -> null
25
```


15.3 uploadFile — Subir archivo a Storage

Función genérica que sube un File al bucket especificado de Supabase Storage. Genera un nombre único combinando timestamp y string aleatorio para evitar colisiones, y devuelve la URL pública del archivo subido.

```
1  const uploadFile = async (file: File, bucket: string) => {
2    // Generar nombre único: timestamp + random + extensión
3    const ext = file.name.split('.').pop()?.toLowerCase();
4    const path = `${Date.now()}-${Math.random().toString(36).slice(2)}-${ext}`;
5
6    // Subir con upsert (sobrescribe si existe)
7    const { error } = await supabase.storage
8      .from(bucket)
9      .upload(path, file, { upsert: true });
10
11    if (error) throw new Error(`Error al subir imagen: ${error.message}`);
12
13    // Obtener URL pública
14    const { data: urlData } = supabase.storage
15      .from(bucket)..getPublicUrl(path);
16
17    return urlData.publicUrl;
18  };
19
20  // Uso: const url = await uploadFile(imageFile, 'recipe-images');
21
22
```

15.4 convertToWebP — Conversión de imagen en cliente

Función que convierte cualquier imagen (Blob) a formato WebP con calidad 0.85 usando un canvas. Opcionalmente redimensiona si supera maxWidth. Se usa para optimizar las miniaturas extraídas de YouTube.

```
1  TYPESCRIPT
2  const convertToWebP = (blob: Blob, maxWidth?: number): Promise<Blob> => {
3    return new Promise((resolve, reject) => {
4      const img = document.createElement('img');
5
6      img.onload = () => {
7        const canvas = document.createElement('canvas');
8        let w = img.width;
9        let h = img.height;
10
11        // Redimensionar manteniendo aspect ratio
12        if (maxWidth && w > maxWidth) {
13          h = Math.round((h * maxWidth) / w);
14          w = maxWidth;
15        }
16
17        canvas.width = w;
18        canvas.height = h;
19        const ctx = canvas.getContext('2d');
20        if (!ctx) { reject(new Error('No canvas context')); return; }
21
22        ctx.drawImage(img, 0, 0, w, h);
23
24        // Convertir a WebP con calidad 0.85
25        canvas.toBlob(
26          (result) => result
27            ? resolve(result)
28            : reject(new Error('Error al convertir a WebP')),
29          'image/webp',
30          0.85
31        );
32      };
33    });
34
35    img.onerror = () => reject(new Error('Error al cargar imagen'));
36    img.crossOrigin = 'anonymous'; // necesario para imágenes externas
37    img.src = URL.createObjectURL(blob);
38  });
39  };
40
41
42
43
```

15.5 Extracción automática de miniaturas de YouTube

Cuando el admin pega una URL de YouTube y no sube una imagen manual, el sistema descarga automáticamente la miniatura del video desde i.ytimg.com y la convierte a WebP. Intenta primero la máxima resolución (maxresdefault) y cae a hqdefault si no existe.

```
1  // Pseudocódigo simplificado del flujo
2  if (youtubeId) {
3    const needsImage = !!image_url && !!imageFile;
4    const needsThumbnail = !!thumbnail_url && !!thumbnailFile;
5
6    if (needsImage || needsThumbnail) {
7      const urls = [
8        `https://img.youtube.com/vi/${youtubeId}/maxresdefault.jpg`,
9        `https://img.youtube.com/vi/${youtubeId}/hqdefault.jpg`,
10     ];
11
12     let blob: Blob | null = null;
13     for (const url of urls) {
14       try {
15         const res = await fetch(url);
16         if (res.ok) {
17           blob = await res.blob();
18           if (blob.size > 5000) break; // válida
19           blob = null;
20         }
21       } catch {
22         continue;
23       }
24     }
25     if (blob) {
26       const webpBlob = await convertToWebP(blob);
27       // Subir imagen principal a Storage
28       // ...
29       // Generar thumb 640px y subir también
30       const thumbBlob = await convertToWebP(blob, 640);
31     } // ...
32   }
33 }
```

16. Panel administrativo

16.1 Acceso

El panel está en /admin. Para acceder, el usuario debe estar autenticado y tener el campo `is_admin = true` en la tabla `profiles`. Cualquier otro usuario es redirigido a / (home).

La protección se aplica en dos niveles: el middleware (redirige a /login si no hay sesión) y el Server Component de la página (verifica `is_admin`).

16.2 Estructura del panel

El panel está construido con un sidebar fijo en desktop (sticky) y tabs horizontales scrollables en móvil. Tiene cuatro pestañas:

Tab	Funcionalidad
Resumen (overview)	Tarjetas con estadísticas: número de recetas, mensajes sin leer, posts publicados.
Recetas	Listado, creación y edición de recetas. Buscador con scoring. Borrado con confirmación.
Mensajes	Mensajes recibidos por el formulario de contacto. Filtros: todos, no leídos, leídos. Al hacer click se marca como leído.
Publicaciones	CRUD de posts del blog: contenido, imágenes múltiples, video de YouTube opcional.

16.3 Flujo de creación de receta

1. Admin hace click en 'Nueva receta' → aparece formulario.
2. Llena título, descripción, ingredientes (uno por línea), pasos (uno por línea), tiempo, porciones y dificultad.
3. Opcionalmente sube imagen principal (WebP) y miniatura (WebP).
4. Opcionalmente pega URL de YouTube. Si no hay imagen, se extrae automáticamente.
5. Al enviar: se valida URL de YouTube si existe, se sube imagen/miniatura, se construye el slug, se inserta en la tabla recipes.
6. `router.refresh()` recarga el listado en el servidor.

16.4 Validación de imágenes WebP

El admin solo permite imágenes en formato WebP para garantizar tamaños pequeños y carga rápida. La validación se hace en el handler del input file:

```
1  const handleImageChange = (e: React.ChangeEvent<HTMLInputElement>) => {
2    const file = e.target.files?.[0];
3    if (!file) { setImageFile(null); return; }
4
5    if (file.type !== 'image/webp') {
6      setMessage('Error: Solo se permiten imágenes en formato WebP (.webp)');
7      e.target.value = ''; // limpiar input
8      setImageFile(null);
9      return;
10   }
11
12   setMessage('');
13   setImageFile(file);
14 };
```

16.5 Carga de datos en servidor

La página `admin/page.tsx` es un Server Component que pre-carga todos los datos antes de enviar el HTML al navegador. Esto evita estados de loading innecesarios en el cliente.

```
1 // app/(main)/admin/page.tsx
2 export default async function AdminPage() {
3   const supabase = await createSupabaseServer();
4   const { data: { user } } = await supabase.auth.getUser();
5
6   if (!user) redirect('/login');
7
8   const { data: profile } = await supabase
9     .from('profiles').select('is_admin')
10    .eq('id', user.id).single();
11
12   if (!profile?.is_admin) redirect('/');
13
14   // Cargar todo en paralelo (3 queries simultáneas)
15   const { data: recipes } = await supabase
16     .from('recipes').select('*')
17     .order('created_at', { ascending: false });
18
19   const { data: messages } = await supabase
20     .from('contact_messages').select('*')
21     .order('created_at', { ascending: false });
22
23   const { data: posts } = await supabase
24     .from('posts').select('*')
25     .order('created_at', { ascending: false });
26
27   return <AdminPanel
28     initialRecipes={recipes ?? []}
29     initialMessages={messages ?? []}
30     initialPosts={posts ?? []}
31     userId={user.id}
32   />;
33 }
34
35
```

17. Rutas y navegación

17.1 Mapa de rutas

Next.js App Router usa el sistema de archivos: cada carpeta es una ruta. Las carpetas entre paréntesis como (main) son grupos de rutas que no afectan la URL pero permiten compartir layouts.

Ruta	Tipo	Función
GET /	Server Component	Home con recetas recientes y stats.
GET /recetas	Server Component	Listado de recetas + buscador (?search=).
GET /recetas/[id]	Server Component	Detalle de receta. id es el slug.
GET /favoritos	Server Component	Favoritos del usuario (requiere login).
GET /perfil	Server Component	Perfil del usuario.
GET /admin	Server Component	Panel admin (requiere is_admin).
GET /blog	Server Component	Listado de posts del blog.
GET /sobre-mi	Server Component	Sobre el chef. Si es admin, permite subir foto.
GET /contacto	Server Component	Formulario de contacto público.
GET /auth/login	Client Component	Login con email + Google.
GET /auth/register	Client Component	Registro con captcha.
GET /auth/callback	Route Handler	Intercambia code por session.
GET /api/search	Route Handler	API JSON de búsqueda con caché.

17.2 Grupo de rutas (main)

La carpeta app/(main)/ agrupa todas las rutas que comparten el layout principal (Navbar + Footer). El paréntesis indica que el nombre no forma parte de la URL final.

17.3 Rutas dinámicas

La carpeta [id] en `app/(main)/recetas/[id]/page.tsx` define un segmento dinámico. El valor entre corchetes se recibe como `params` en el componente:

```
1 // app/(main)/recetas/[id]/page.tsx
2 interface Props {
3   params: Promise<{ id: string }>;
4 }
5
6 export default async function RecipeDetailPage({ params }: Props) {
7   const { id: slug } = await params;
8   // Buscar receta por slug (no por UUID)
9   const { data: recipe } = await supabase
10     .from('recipes').select('...').eq('slug', slug).single();
11   // ...
12 }
```

17.4 Server Components vs Client Components

Por defecto, todos los componentes son Server Components (se renderizan en el servidor, no envían JavaScript al cliente). Para hacer un componente interactivo (con `useState`, `onClick`, `useEffect`), debe llevar la directiva `'use client'` al inicio del archivo.

Tipo	Componentes
Server Component	page.tsx (todas), RecipeCard, Footer. Datos directos de Supabase, sin loading states.
Client Component	Navbar, LikeButton, FavoriteButton, CommentSection, LiveSearchBar, AdminPanel, formularios de auth.

18. Guía de instalación y despliegue

18.1 Preparación del entorno local

18.1.1 Clonar el repositorio

```
git clone https://github.com/usuario/cocina-project.git
cd cocina-project
```

18.1.2 Instalar dependencias

```
npm install
```

18.2 Configuración de Supabase

18.2.1 Crear el proyecto

1. Acceder a <https://supabase.com/dashboard>.
2. Hacer click en 'New Project'.
3. Elegir un nombre descriptivo (por ejemplo, 'mi-cocina-prod').
4. Seleccionar región geográfica cercana y establecer contraseña para la base de datos.
5. Esperar aproximadamente dos minutos a que se aprovisione.

18.2.2 Obtener las credenciales

En el panel del proyecto, ir a Settings → API y copiar:

- Project URL → variable NEXT_PUBLIC_SUPABASE_URL
- anon public key → variable NEXT_PUBLIC_SUPABASE_ANON_KEY

18.2.3 Archivo .env.local

En la raíz del proyecto, crear un archivo .env.local con el siguiente contenido:

```
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOi...
```

```
# Cloudflare Turnstile (opcional)
```

```
NEXT_PUBLIC_TURNSTILE_SITE_KEY=0x4AAAAAA...
TURNSTILE_SECRET_KEY=0x4AAAAAA...
```

18.2.4 Ejecutar el esquema SQL

1. En Supabase, ir a SQL Editor y hacer click en 'New query'.
2. Copiar todo el contenido del archivo supabase-schema.sql.
3. Hacer click en 'Run'. El script debe ejecutarse sin errores.
4. Verificar en Table Editor que se hayan creado las doce tablas del esquema.

18.2.5 Configurar proveedores de autenticación

Para desarrollo, es recomendable desactivar la confirmación obligatoria de correo:

1. Ir a Authentication → Providers → Email.
2. Desactivar 'Confirm email'.
3. Guardar los cambios.

Para habilitar Google OAuth:

1. Crear credenciales OAuth en Google Cloud Console.
2. En Supabase: Authentication → Providers → Google.
3. Pegar Client ID y Client Secret, habilitar el proveedor.

18.3 Primer administrador

Una vez la aplicación esté corriendo, el primer usuario debe registrarse normalmente y luego ser promovido manualmente a administrador mediante SQL:

```
UPDATE public.profiles
SET is_admin = true
WHERE email = 'tu@email.com';
```

18.4 Ejecución en desarrollo

```
# Limpiar caché (por si acaso)
rm -rf .next

# Iniciar servidor de desarrollo
npm run dev
```

La aplicación estará disponible en <http://localhost:3000>.

18.5 Despliegue en producción

La aplicación está optimizada para desplegarse en Vercel, pero es compatible con cualquier plataforma que soporte Next.js (Netlify, AWS Amplify, servidores Node propios).

18.5.1 Despliegue en Vercel

1. Importar el repositorio desde el panel de Vercel.
2. Configurar las variables de entorno del archivo `.env.local`.
3. Vercel detectará automáticamente Next.js y desplegará la aplicación.

18.5.2 Compilación manual

```
npm run build
```

```
npm start
```

19. Estilos y diseño

19.1 Paleta de colores: Warm Linen

El sistema usa una paleta cálida personalizada definida en `tailwind.config.js`. Los nombres están en inglés y reflejan ingredientes y materiales de cocina.

Color	Uso
linen (#F5EFE6)	Fondo principal, color cálido.
parchment (#EDE0CC)	Fondo de cards y elementos secundarios.
walnut (#5C3D2E)	Texto principal, color oscuro.
paprika (#B5451B)	Color de acción (botones, enlaces, acentos).
herb (#4A7C59)	Dificultad fácil, elementos frescos.
honey (#C9922A)	Dificultad media, acento dorado.
wine (#7A2B32)	Dificultad difícil, color de likes.

Existen alias semánticos (cream → linen, charcoal → walnut, terra → paprika, sage → herb, gold → honey) para no romper código antiguo durante migraciones de diseño.

19.2 Tipografías

Familia	Uso
display (Lora)	Títulos, nombres de recetas, encabezados de blog. Serif.
ui (Poppins)	Navbar, botones, badges, labels, stats. Sans-serif geométrica.
body (Inter)	Cuerpo de texto, ingredientes, comentarios. Sans-serif optimizada para lectura.

19.3 Clases utilitarias del proyecto

Estas clases aparecen repetidamente en el código y vale la pena conocerlas:

Clase	Función
input-field	Estilo unificado para inputs y textareas.
btn-primary	Botón principal con color paprika.
card	Tarjeta con sombra suave y bordes redondeados.
animate-fade-in	Aplica animación fadeIn al elemento.
animate-scale-in	Aplica animación scaleIn (útil en dropdowns).

20. Mantenimiento y solución de problemas

20.1 Problemas frecuentes

20.1.1 La sesión no persiste o se cierra sola

Verificar que middleware.ts esté ubicado en la raíz del proyecto, al mismo nivel que package.json. Si se encuentra dentro de otra carpeta, Next.js no lo ejecutará y la sesión expirará silenciosamente. Solución complementaria: limpiar la caché con `rm -rf .next` y reiniciar.

20.1.2 Error 'relation does not exist'

Indica que el esquema SQL no se ejecutó correctamente. Abrir Supabase Dashboard, ir a Table Editor y verificar que existan las doce tablas. Si faltan, volver a ejecutar el archivo supabase-schema.sql completo.

20.1.3 Las imágenes no cargan

Revisar en Supabase → Storage que los buckets recipe-images, recipe-videos, post-images y site-assets existan y estén marcados como públicos. Si fueron creados pero son privados, sus URLs retornarán error 403.

20.1.4 Error 401 / No autenticado

Confirmar que las variables en .env.local tengan el prefijo NEXT_PUBLIC_ correcto. Sin ese prefijo, Next.js no las expone al código del navegador. Después de modificar .env.local siempre es necesario reiniciar el servidor.

20.1.5 Fallo al publicar recetas

Revisar que el usuario tenga el flag `is_admin = true`. Puede verificarse con la consulta:

```
SELECT id, email, is_admin
FROM public.profiles
WHERE email = 'tu@email.com';
```

20.2 Tareas de mantenimiento periódicas

- **Revisión de mensajes de contacto:** al menos semanalmente, marcar como leídos o eliminar los obsoletos.
- **Limpieza de archivos huérfanos:** imágenes y videos en Storage que ya no están referenciados por ninguna receta pueden acumularse. Se recomienda una auditoría trimestral.
- **Actualización de dependencias:** revisar mensualmente con npm outdated y aplicar parches de seguridad con npm audit fix.
- **Backup de base de datos:** Supabase incluye backups automáticos diarios en los planes de pago. Verificar su disponibilidad en Database → Backups.
- **Monitoreo de uso:** revisar en Supabase Dashboard el consumo de ancho de banda, almacenamiento y operaciones.

20.3 Logs y depuración

Para depurar problemas, las siguientes fuentes son útiles:

- **Consola del navegador:** errores de cliente, fallos de autenticación, problemas de Realtime.
- **Terminal de desarrollo:** logs del servidor Next.js, errores de Server Components, fallos de compilación.
- **Supabase → Logs Explorer:** consultas SQL, errores de autenticación, actividad de API.
- **Supabase → Database → Query Performance:** consultas lentas que pueden beneficiarse de un índice.

21 Comandos de diagnóstico Supabase

```
1  -- Verificar que tu perfil existe y es admin
2  SELECT id, email, is_admin FROM public.profiles;
3
4  -- Ver políticas activas en una tabla
5  SELECT * FROM pg_policies WHERE tablename = 'recipes';
6
7  -- Ver buckets de storage
8  SELECT * FROM storage.buckets;
9
10 -- Probar insertar una receta manualmente
11 INSERT INTO public.recipes (
12     title, slug, description, ingredients, steps,
13     prep_time, difficulty, author_id
14 ) VALUES (
15     'Test!', 'test-receta', 'prueba',
16     '{"harina"}', '{"mezclar"}', 10, 'fácil',
17     '<tu-uuid>'
18 );
19
20 -- Si necesitas hacer admin a otro usuario
21 UPDATE public.profiles
22 SET is_admin = true
23 WHERE email = 'usuario@ejemplo.com';
24
26
```

22. Glosario técnico

Término	Definición
App Router	Sistema de enrutamiento basado en carpetas introducido en Next.js 13, que soporta Server Components y layouts anidados.
BaaS	Backend as a Service. Modelo de servicio cloud que provee base de datos, autenticación y almacenamiento listos para usar, eliminando la necesidad de operar un backend propio.
CAPTCHA	Prueba automatizada para distinguir humanos de bots. En este proyecto se utiliza Cloudflare Turnstile como alternativa invisible al reCAPTCHA tradicional.
CRUD	Create, Read, Update, Delete. Conjunto de operaciones básicas sobre datos persistentes.
HTTP-only cookie	Cookie que no es accesible desde JavaScript del cliente, mitigando ataques XSS. Usada por Supabase para almacenar tokens de sesión.
OAuth	Protocolo de autorización que permite iniciar sesión en un servicio mediante credenciales de otro (por ejemplo, Google).
Optimistic update	Patrón de UI que refleja inmediatamente la acción del usuario (como dar like) antes de confirmar con el servidor. Si falla, se revierte.
Realtime	Funcionalidad de Supabase que permite suscribirse a cambios en la base de datos vía WebSockets, recibiendo eventos INSERT/UPDATE/DELETE en vivo.
RLS	Row Level Security. Mecanismo de PostgreSQL que filtra las filas visibles y modificables de una tabla según políticas declarativas basadas en el usuario autenticado.
RSC	React Server Components. Componentes que se renderizan exclusivamente en el servidor, reduciendo el JavaScript enviado al navegador.
Slug	Identificador legible usado en URLs (por ejemplo, tacos-al-pastor en lugar de un UUID).

Término	Definición
Trigger	Función de base de datos que se ejecuta automáticamente ante un evento (INSERT, UPDATE, DELETE). En este proyecto, <code>handle_new_user</code> crea el perfil al registrarse un usuario.
UUID	Universally Unique Identifier. Identificador de 128 bits usado como clave primaria en todas las tablas para evitar colisiones.
WebP	Formato de imagen moderno desarrollado por Google, con mejor compresión que JPEG/PNG. Usado como formato único en el sistema.

— Fin del manual técnico —